Lab Assignment Week 14

CSC 3320 – System-level Programming

Thursday, April 23rd, 2026

Introduction

Welcome to the 14th programming lab of CSC 3320! Today, we will be covering the following topics:

1. Process Creation
    - fork()
    - execl()

Lab Policies

- Attendance is mandatory.
- Labs must be completed **individually.**
- TAs are here to help you. Ask them for help!
- Lab assignments are due at midnight on the day of your lab.

Deliverables:

1. The C Code for your program. (.c file).
2. A screenshot of the output in the Terminal.

If you have any questions, please do not hesitate to ask your TA.

Program: Building a Simple Command Launcher

In Unix-like operating systems (like Linux or macOS), the fork() system call is the primary way to create a new process. When fork() is called, the operating system creates a nearly identical copy of the calling process (the parent). The new process is called the child. Both processes continue execution from the point right after the fork() call, but they can determine whether they are the parent or the child by checking the return value of fork().

For this lab, you must write a C program named my_shell that simulates a very basic command shell. Just like the program that I demonstrated in class. It should:

1. Display a simple prompt (e.g., my_shell :>).
2. Read a line of input from the user. This line will represent a command (e.g., ls -l).
3. If the user types "exit", the program should terminate gracefully.
4. For any other input, the program should:
    a. Parse the input line into a command (the first word) and concatenate that command to the path where the UNIX shell executables are stored /bin/.
    b. Use fork() to create a child process.
        i. **In the child process:** Use execl() to attempt to execute the command. Remember that execl() searches the system using the path argument to find the executable. If execl() fails, the child should print an informative error message (e.g., using perror() or printf()) and exit.
        ii. **In the parent process:** Use wait() to wait for the child process to complete. After the child terminates, the parent should loop back to display the prompt and wait for the next command.
5. Include basic error handling for fork(), execl(), and wait(). If a system call fails, print an error message using perror() or printf().

Hints:

- Headers: You will need the following header (library) files.
    - <stdio.h > (for standard I/O like printf, fgets, perror)
    - <stdlib.h> (for exit)
    - <unistd.h> (for fork, execl, wait)
    - <string.h> (for strcmp, strncat, strlen)
- Prompt & Input: Inside the loop, print the prompt and use fgets() to read a line of input from the user into a character buffer. Remember, fgets() includes the newline character (\n) if there's space; you'll likely want to remove it. Check for fgets errors or end-of-file.

Example Output

```
[ssaghaeiannejadesfa1@gsuad.gsu.edu@snowball unix_demo]$ ./my_sell

my_shell :>ls
class_notes.txt  hello  hello_world.c  lab_1.txt  my_sell  my_shell.c  sample.txt  test
test.c  typescript.log  week2

my_shell :>cal
     April 2026
Su Mo Tu We Th Fr Sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30


my_shell :>date
Thu Apr 23 00:29:41 EDT 2026

my_shell :>exit
[ssaghaeiannejadesfa1@gsuad.gsu.edu@snowball unix_demo]$
```

Deliverables

For today's lab, you will need to upload the C program code for your simple command launcher program and its output in the terminal on iCollege. Please name your C code and screenshot as follows:

- C Files
  - lastname_firstname_filename.c
  - For example: **turing_alan_my_shell.c**
- Screenshots
  - lastname_firstname_filename.png
  - For example: **turing_alan_my_shell.png**